

Why an Augmented Lagrangian for our PINN training

Fixing loss-balancing drift at the source

PINN Option-Pricing project

June 3, 2026

1. The problem: our loss is a “penalty method”

Today we minimise a **weighted sum** with auto-tuned weights (grad-norm):

$$\mathcal{L} = \lambda_{\text{pde}}\mathcal{L}_{\text{pde}} + \lambda_{\text{tc}}\mathcal{L}_{\text{tc}} + \lambda_{\text{bc}}\mathcal{L}_{\text{bc}} + \lambda_{\text{data}}\mathcal{L}_{\text{data}}.$$

In optimisation terms this is the **penalty method** for a constrained problem.

Its built-in flaw

To truly *enforce* a term you must send its weight $\rightarrow \infty$; large weights make the optimisation stiff and unstable.

What we observe

λ_{data} runs away to $\sim 10^5$, training drifts: physics-only RMSE \$8.76 $\rightarrow$ hybrid \$10+, and no-arbitrage violations explode. The weights never settle.

2. Reframe: it is really a *constrained* problem

We don't want a blend of physics and data. We want:

minimise the market-data misfit subject to obeying the Black–Scholes PDE, the boundary/terminal conditions, and no-arbitrage.

$$\begin{aligned} \min_{\theta} \quad & \mathcal{L}_{\text{data}}(\theta) && \text{(objective: fit the market)} \\ \text{s.t.} \quad & \mathcal{L}_{\text{pde}}(\theta) = 0, \mathcal{L}_{\text{tc}}(\theta) = 0, \mathcal{L}_{\text{bc}}(\theta) = 0 && \text{(equality constraints)} \\ & \underbrace{\text{ReLU}(-\partial_{mm}^2 \hat{v})}_{\text{butterfly}} \leq 0, \underbrace{\text{ReLU}(-\partial_{\tau} \hat{v})}_{\text{calendar}} \leq 0 && \text{(no-arbitrage)} \end{aligned}$$

The key shift

Data is the **objective**; physics and no-arbitrage are **constraints**. The weighted sum throws this structure away by treating them as co-equal losses.

3. What the Augmented Lagrangian does

Introduce a **multiplier** μ_k per constraint c_k and add a penalty:

$$\mathcal{L}_A(\theta, \mu; \rho) = \mathcal{L}_{\text{data}}(\theta) + \sum_k \mu_k c_k(\theta) + \frac{\rho}{2} \sum_k c_k(\theta)^2.$$

Alternate two cheap steps:

- descend on θ (a few gradient steps);
- update multipliers $\mu_k \leftarrow \mu_k + \rho c_k$ (inequalities: $\mu_k \leftarrow \max(0, \mu_k + \rho c_k)$).

Why it cures the drift

The multiplier **accumulates the violation** and does the job that $\rho \rightarrow \infty$ would do — but with a **fixed, moderate** ρ . It is **self-limiting**: once $c_k \approx 0$ the update adds ≈ 0 , so μ_k stops growing. No runaway, by construction.

Analogy: penalty method = ever-larger fines until compliance ($\rightarrow \infty$); augmented Lagrangian = a **thermostat** that settles at exactly the pressure needed, then holds.

4. What it enables for us

- **Fixes drift at the source** $\Rightarrow$ trajectory converges $\Rightarrow$ final-epoch reliable $\Rightarrow$ *we can drop the validation set* and grow the training window.
- **Encodes the right structure**: physics & no-arbitrage are constraints, data is the objective.
- **Unifies no-arbitrage**: just another inequality constraint, active only where violated — self-modulating.
- **Scales to Stage 2**: σ -regularisation and the second network are simply more constraints.

		uses
current grad-norm	augmented Lagrangian	per-term gradient <i>norms</i> constraint <i>values</i>

Cost

One robust knob ρ . Multiplier updates use constraint *values*, not per-term gradients — so it is **cheaper and more stable** than what we run now.

PINN precedent: PECANN (Basir & Senocak, 2022); AL-PINN (Son et al., 2022). Method: Hestenes & Powell, 1969.